

# **Competitive Programming**

**Section 21 Editorial: Graphs I Mixed Contest**

Competitive Programming Course

2026-07-28

# Table of contents

|          |                                       |           |
|----------|---------------------------------------|-----------|
| <b>1</b> | <b>Category coverage</b>              | <b>5</b>  |
| <b>2</b> | <b>Editorial overview</b>             | <b>6</b>  |
| <b>3</b> | <b>Tool reminders</b>                 | <b>7</b>  |
| 3.1      | DSU and Kruskal . . . . .             | 7         |
| 3.2      | Kahn topological sort . . . . .       | 7         |
| 3.3      | 0-1 BFS . . . . .                     | 7         |
| <b>4</b> | <b>A. Existing Network</b>            | <b>8</b>  |
| 4.1      | Restatement . . . . .                 | 8         |
| 4.2      | Constraints and traps . . . . .       | 8         |
| 4.3      | Observations . . . . .                | 8         |
| 4.4      | Sample walkthrough . . . . .          | 8         |
| 4.5      | Algorithm . . . . .                   | 9         |
| 4.6      | Correctness proof . . . . .           | 10        |
| 4.7      | Complexity . . . . .                  | 10        |
| 4.8      | C++ implementation notes . . . . .    | 10        |
| 4.9      | Python implementation notes . . . . . | 10        |
| 4.10     | Full C++ solution . . . . .           | 11        |
| 4.11     | Full Python solution . . . . .        | 12        |
| 4.12     | Common wrong answers . . . . .        | 13        |
| 4.13     | Tests that matter . . . . .           | 13        |
| <b>5</b> | <b>B. Build Timeline</b>              | <b>14</b> |
| 5.1      | Restatement . . . . .                 | 14        |
| 5.2      | Constraints and traps . . . . .       | 14        |
| 5.3      | Observations . . . . .                | 14        |
| 5.4      | Sample walkthrough . . . . .          | 14        |
| 5.5      | Algorithm . . . . .                   | 15        |
| 5.6      | Correctness proof . . . . .           | 16        |
| 5.7      | Complexity . . . . .                  | 16        |
| 5.8      | C++ implementation notes . . . . .    | 17        |
| 5.9      | Python implementation notes . . . . . | 17        |
| 5.10     | Full C++ solution . . . . .           | 17        |
| 5.11     | Full Python solution . . . . .        | 18        |
| 5.12     | Common wrong answers . . . . .        | 19        |
| 5.13     | Tests that matter . . . . .           | 19        |
| <b>6</b> | <b>C. One-Way Reversals</b>           | <b>20</b> |
| 6.1      | Restatement . . . . .                 | 20        |
| 6.2      | Constraints and traps . . . . .       | 20        |
| 6.3      | Observations . . . . .                | 20        |
| 6.4      | Sample walkthrough . . . . .          | 20        |
| 6.5      | Algorithm . . . . .                   | 21        |

|           |                                                  |           |
|-----------|--------------------------------------------------|-----------|
| 6.6       | Correctness proof . . . . .                      | 22        |
| 6.7       | Complexity . . . . .                             | 22        |
| 6.8       | C++ implementation notes . . . . .               | 22        |
| 6.9       | Python implementation notes . . . . .            | 23        |
| 6.10      | Full C++ solution . . . . .                      | 23        |
| 6.11      | Full Python solution . . . . .                   | 24        |
| 6.12      | Common wrong answers . . . . .                   | 24        |
| 6.13      | Tests that matter . . . . .                      | 25        |
| <b>7</b>  | <b>Testing and stress strategy</b>               | <b>26</b> |
| <b>8</b>  | <b>What to carry forward</b>                     | <b>27</b> |
| <b>9</b>  | <b>D. Reliable Link</b>                          | <b>28</b> |
| 9.1       | Restatement . . . . .                            | 28        |
| 9.2       | Threshold viewpoint . . . . .                    | 28        |
| 9.3       | Walk-through . . . . .                           | 28        |
| 9.4       | Algorithm . . . . .                              | 29        |
| 9.5       | Correctness proof . . . . .                      | 29        |
| 9.6       | Complexity . . . . .                             | 29        |
| 9.7       | Implementation notes . . . . .                   | 30        |
| 9.8       | Full C++ solution . . . . .                      | 30        |
| 9.9       | Full Python solution . . . . .                   | 31        |
| 9.10      | Common wrong answers . . . . .                   | 32        |
| 9.11      | Tests that matter . . . . .                      | 32        |
| <b>10</b> | <b>Stress testing</b>                            | <b>33</b> |
| <b>11</b> | <b>What to carry forward</b>                     | <b>34</b> |
| <b>12</b> | <b>E. Station Coverage Report</b>                | <b>35</b> |
| 12.1      | Restatement and model . . . . .                  | 35        |
| 12.2      | Multi-source BFS . . . . .                       | 35        |
| 12.3      | Algorithm . . . . .                              | 35        |
| 12.4      | Correctness proof . . . . .                      | 35        |
| 12.5      | Complexity and pitfalls . . . . .                | 36        |
| 12.6      | Tests that matter . . . . .                      | 36        |
| 12.7      | Full C++ solution . . . . .                      | 36        |
| 12.8      | Full Python solution . . . . .                   | 37        |
| <b>13</b> | <b>F. Component Repair</b>                       | <b>39</b> |
| 13.1      | Restatement and observations . . . . .           | 39        |
| 13.2      | Algorithm . . . . .                              | 39        |
| 13.3      | Correctness proof . . . . .                      | 39        |
| 13.4      | Complexity and pitfalls . . . . .                | 40        |
| 13.5      | Tests that matter . . . . .                      | 40        |
| 13.6      | Full C++ solution . . . . .                      | 40        |
| 13.7      | Full Python solution . . . . .                   | 41        |
| <b>14</b> | <b>G. Alternating Road Route</b>                 | <b>42</b> |
| 14.1      | Why a vertex alone is not enough state . . . . . | 42        |
| 14.2      | Expanded graph and algorithm . . . . .           | 42        |
| 14.3      | Correctness proof . . . . .                      | 42        |

|                                               |    |
|-----------------------------------------------|----|
| 14.4 Complexity and common mistakes . . . . . | 43 |
| 14.5 Tests that matter . . . . .              | 43 |
| 14.6 Full C++ solution . . . . .              | 43 |
| 14.7 Full Python solution . . . . .           | 44 |

# 1 Category coverage

| Category                          | Coverage problem           |
|-----------------------------------|----------------------------|
| BFS and unweighted shortest paths | E. Station Coverage Report |
| DFS, components, and cycles       | F. Component Repair        |
| topological sorting and DAG DP    | B. Build Timeline          |
| DSU and MST                       | A. Existing Network        |
| Dijkstra and weighted modeling    | G. Alternating Road Route  |
| 0-1 BFS                           | C. One-Way Reversals       |

The additional problems are self-contained local packages and are included by this section's checker. D. Reliable Link remains an extra DSU threshold problem rather than being counted as the Dijkstra representative.

## 2 Editorial overview

This section is a recognition drill for the first graph phase.

| Problem              | Intended model                                 | Algorithm     |
|----------------------|------------------------------------------------|---------------|
| A. Existing Network  | free components plus paid undirected edges     | DSU + Kruskal |
| B. Build Timeline    | dependency DAG with longest prerequisite chain | Kahn + DP     |
| C. One-Way Reversals | directed roads become 0/1 weighted edges       | 0-1 BFS       |

The most important editorial habit is to separate:

```
modeling argument
algorithm correctness
implementation details
```

## 3 Tool reminders

### 3.1 DSU and Kruskal

DSU supports:

```
find(x): component representative
union(a,b): merge components if they are different
```

Kruskal:

```
sort edges by increasing weight
for each edge:
    if its endpoints are in different components:
        take it and union the components
```

The algorithm minimizes the total cost of connecting all components because at each cut between already-separated components, taking the cheapest safe edge is valid.

### 3.2 Kahn topological sort

Maintain indegrees. Vertices with indegree zero currently have no unmet prerequisites. Pop them, remove their outgoing edges, and push new indegree-zero vertices.

If the algorithm processes fewer than  $n$  vertices, some directed cycle remains.

### 3.3 0-1 BFS

For edge costs 0 and 1:

```
cost 0 relaxation -> push_front
cost 1 relaxation -> push_back
```

This maintains the same distance order that Dijkstra would produce for these restricted weights.

## 4 A. Existing Network

### 4.1 Restatement

There are  $n$  cities. Some undirected cables already exist and cost nothing. You may buy additional undirected cables, each with a nonnegative cost. Find the minimum total extra cost needed so all cities are connected. If no set of paid cables can connect all cities, print IMPOSSIBLE.

### 4.2 Constraints and traps

$n$ , free edges, and paid edges can each be as large as  $2 * 10^5$ , so an  $O(n^2)$  approach is impossible.

Important traps:

```
Free edges are real connections and must be merged first.  
The objective is to connect all cities, not to find a shortest route.  
The answer can exceed 32-bit int, so use long long / Python int.  
The graph may remain disconnected even after all paid edges are considered.
```

### 4.3 Observations

After using all free cables, the cities are partitioned into connected components. Inside one free component, no paid cable is needed.

So the problem becomes:

```
Given existing components, buy minimum-cost paid edges that connect them.
```

This is exactly Kruskal's MST idea, except that Kruskal starts after the free edges have already been unioned.

### 4.4 Sample walkthrough

Input:

```
5 2 4
1 2
4 5
2 3 4
3 4 7
1 5 20
2 5 10
```

Free unions:

```
{1,2}, {3}, {4,5}
```

Paid edges sorted:

```
2--3 cost 4
3--4 cost 7
2--5 cost 10
1--5 cost 20
```

Kruskal decisions:

```
take 2--3 cost 4    -> {1,2,3}, {4,5}
take 3--4 cost 7    -> {1,2,3,4,5}
stop
```

Answer:  $4 + 7 = 11$ .

## 4.5 Algorithm

```
read n, f, p
create DSU with n singleton components
components = n

for each free edge u, v:
    if union(u, v):
        components -= 1

read all paid edges as (weight, u, v)
sort paid edges by weight
answer = 0

for each paid edge in increasing weight:
    if components == 1:
        break
    if union(u, v):
        answer += weight
        components -= 1
```

```
if components == 1: print answer
else: print IMPOSSIBLE
```

## 4.6 Correctness proof

**Lemma 1.** After all free edges are unioned, each DSU component is exactly one connected component of the graph made only from free edges.

**Proof.** DSU union merges the endpoints of every free edge, so all vertices connected by a free-edge path end in the same DSU component. DSU never merges vertices except through a free edge, so vertices in different free connected components remain separated.  $\square$

**Lemma 2.** Running Kruskal on paid edges after the free unions produces a minimum-cost set of paid edges that connects the free components, if such a set exists.

**Proof.** Contract every free component into one super-vertex. A paid edge between two cities becomes an edge between their super-vertices; paid edges inside the same super-vertex are useless. Kruskal's algorithm on this contracted undirected graph returns a minimum spanning tree when the graph is connected. The DSU implementation after free unions is exactly Kruskal on that contracted graph.  $\square$

**Theorem.** The algorithm prints the minimum additional cost to connect all cities, or IMPOSSIBLE if it cannot be done.

**Proof.** By Lemma 1, the initial DSU state correctly represents the zero-cost connectivity. By Lemma 2, the paid edges selected afterward have minimum total cost among all ways to connect those components. If after processing all paid edges more than one component remains, no paid edge set can connect the contracted graph, so connecting all cities is impossible.  $\square$

## 4.7 Complexity

Sorting paid edges dominates:

```
Time:  $O((f + p) \alpha(n) + p \log p)$ 
Memory:  $O(n + p)$ 
```

Here  $\alpha(n)$  is the inverse Ackermann factor from DSU, effectively constant in contest constraints.

## 4.8 C++ implementation notes

Store paid edges as `(weight, u, v)` so normal tuple sorting sorts by weight first. Use `long long` for weights and total cost.

## 4.9 Python implementation notes

Use an iterative `find` with path compression. Python's integers are unbounded, so no special type is needed for the answer.

## 4.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> parent, sz;
    DSU(int n) : parent(n), sz(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }
    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        parent[b] = a;
        sz[a] += sz[b];
        return true;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, f, p;
    if (!(cin >> n >> f >> p)) return 0;
    DSU dsu(n);
    int components = n;
    for (int i = 0; i < f; i++) {
        int u, v;
        cin >> u >> v;
        if (dsu.unite(u - 1, v - 1)) components--;
    }
    vector<tuple<long long, int, int>> edges;
    for (int i = 0; i < p; i++) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        edges.push_back({w, u - 1, v - 1});
    }
    sort(edges.begin(), edges.end());
    long long total = 0;
```

```

for (auto [w, u, v] : edges) {
    if (components == 1) break;
    if (dsu.unite(u, v)) {
        total += w;
        components--;
    }
}
if (components == 1) cout << total << '\n';
else cout << "IMPOSSIBLE\n";
return 0;
}

```

## 4.11 Full Python solution

```

import sys

class DSU:
    def __init__(self, n: int) -> None:
        self.parent = list(range(n))
        self.size = [1] * n

    def find(self, x: int) -> int:
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a: int, b: int) -> bool:
        a = self.find(a)
        b = self.find(b)
        if a == b:
            return False
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]
        return True

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    f = next(it)
    p = next(it)

```

```

dsu = DSU(n)
components = n
for _ in range(f):
    u = next(it) - 1
    v = next(it) - 1
    if dsu.union(u, v):
        components -= 1
edges = []
for _ in range(p):
    u = next(it) - 1
    v = next(it) - 1
    w = next(it)
    edges.append((w, u, v))
edges.sort()
total = 0
for w, u, v in edges:
    if components == 1:
        break
    if dsu.union(u, v):
        total += w
        components -= 1
print(total if components == 1 else "IMPOSSIBLE")

if __name__ == "__main__":
    main()

```

## 4.12 Common wrong answers

Sorting all edges but forgetting to include free connectivity first.  
 Adding a paid edge even when both endpoints are already in the same component.  
 Printing the partial cost when the graph remains disconnected.  
 Using int for the total cost in C++.

## 4.13 Tests that matter

Already connected using only free edges -> answer 0.  
 No paid edge can connect all components -> IMPOSSIBLE.  
 Multiple paid edges between same components -> choose the cheapest useful one.  
 Large weights -> verifies long long.

## 5 B. Build Timeline

### 5.1 Restatement

There are  $n$  modules. Module  $i$  takes  $\text{time}[i]$  days. A dependency  $a \ b$  means module  $a$  must finish before module  $b$  may start. For each query module, print the earliest day it can finish. If the dependency graph contains a cycle, print IMPOSSIBLE.

### 5.2 Constraints and traps

$n$ ,  $m$ , and  $q$  can be up to  $2 * 10^5$ , so we need linear graph processing.

Important traps:

```
A dependency edge is directed.
Cycles make the whole schedule impossible.
Earliest start is the maximum finish time among prerequisites.
Many queries share the same graph, so compute all finish times once.
Finish times can exceed int.
```

### 5.3 Observations

If there is a cycle, every module on that cycle waits, directly or indirectly, for itself. No valid schedule exists.

If the graph is acyclic, process modules in topological order. When module  $u$  is processed, its earliest finish time is known. For every edge  $u \rightarrow v$ ,  $v$  cannot start before  $\text{finish}[u]$ , so:

```
start[v] = max(start[v], finish[u])
```

After all incoming edges of  $v$  have been processed,  $\text{start}[v]$  contains the maximum finish time of all prerequisites.

### 5.4 Sample walkthrough

Input:

```
5 5 3
3 2 4 6 1
1 3
2 3
3 4
2 5
5 4
3 4 5
```

Initial zero-indegree modules:

```
1 and 2
```

Process:

```
finish[1] = 3
    start[3] = max(0, 3) = 3

finish[2] = 2
    start[3] = max(3, 2) = 3
    start[5] = max(0, 2) = 2

finish[3] = 3 + 4 = 7
    start[4] = max(0, 7) = 7

finish[5] = 2 + 1 = 3
    start[4] = max(7, 3) = 7

finish[4] = 7 + 6 = 13
```

Queries 3 4 5 give:

```
7 13 3
```

## 5.5 Algorithm

```
read n, m, q
read duration[1..n]
build adjacency list and indegree array
read queries

queue = all vertices with indegree 0
start[v] = 0 for all v
processed = 0

while queue not empty:
    u = pop queue
    processed += 1
```

```

    finish[u] = start[u] + duration[u]
    for each u -> v:
        start[v] = max(start[v], finish[u])
        indegree[v] -= 1
        if indegree[v] == 0:
            push v

if processed < n:
    print IMPOSSIBLE
else:
    print finish[x] for each query x

```

## 5.6 Correctness proof

**Lemma 1.** Every vertex popped by Kahn’s algorithm is popped only after all its prerequisites have been popped.

**Proof.** A vertex enters the queue only when its indegree becomes zero. Its indegree is decremented once for each incoming edge after that predecessor is popped. Therefore all incoming neighbors have already been processed before the vertex is popped.  $\square$

**Lemma 2.** When vertex  $v$  is popped,  $\text{start}[v]$  equals the earliest possible start time of module  $v$ .

**Proof.** By Lemma 1, all prerequisites of  $v$  have already been processed. Each prerequisite  $u$  updated  $\text{start}[v]$  with  $\text{finish}[u]$ , so  $\text{start}[v]$  is at least the finish time of every prerequisite. The algorithm uses the maximum of those values, which is exactly the earliest time when all prerequisites have finished.  $\square$

**Lemma 3.** If Kahn’s algorithm processes fewer than  $n$  vertices, the graph contains a directed cycle.

**Proof.** If processing stops early, every remaining vertex has positive indegree within the remaining subgraph. Starting from any remaining vertex and following an incoming edge forever must eventually repeat a vertex, forming a directed cycle.  $\square$

**Theorem.** The algorithm prints `IMPOSSIBLE` exactly for cyclic dependency graphs; otherwise it prints the earliest finish time for each queried module.

**Proof.** By Lemma 3, an early stop identifies a directed cycle, which makes a valid schedule impossible. If all vertices are processed, Lemma 2 gives the earliest start time for each module when it is popped, so  $\text{finish}[v] = \text{start}[v] + \text{duration}[v]$  is the earliest finish time. Queries read from these values.  $\square$

## 5.7 Complexity

```

Time:   $O(n + m + q)$ 
Memory:  $O(n + m + q)$ 

```

## 5.8 C++ implementation notes

Use `queue<int>` for Kahn's algorithm. Store `duration`, `start`, and `finish` as `long long`.

## 5.9 Python implementation notes

Use `collections.deque` as the queue. Reading the entire input at once keeps the parser fast enough for the largest cases.

## 5.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, qn;
    if (!(cin >> n >> m >> qn)) return 0;
    vector<long long> duration(n);
    for (long long &x : duration) cin >> x;
    vector<vector<int>> adj(n);
    vector<int> indeg(n, 0);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a - 1].push_back(b - 1);
        indeg[b - 1]++;
    }
    vector<int> queries(qn);
    for (int &x : queries) {
        cin >> x;
        --x;
    }
    vector<long long> start(n, 0), finish = duration;
    queue<int> q;
    for (int i = 0; i < n; i++) if (indeg[i] == 0) q.push(i);
    int processed = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        processed++;
        finish[u] = start[u] + duration[u];
        for (int v : adj[u]) {
            start[v] = max(start[v], finish[u]);
            indeg[v]--;
        }
    }
```

```

        if (indeg[v] == 0) q.push(v);
    }
}
if (processed != n) {
    cout << "IMPOSSIBLE\n";
} else {
    for (int i = 0; i < qn; i++) {
        if (i) cout << ' ';
        cout << finish[queries[i]];
    }
    cout << '\n';
}
return 0;
}

```

## 5.11 Full Python solution

```

from collections import deque
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    qn = next(it)
    duration = [next(it) for _ in range(n)]
    adj = [[] for _ in range(n)]
    indeg = [0] * n
    for _ in range(m):
        a = next(it) - 1
        b = next(it) - 1
        adj[a].append(b)
        indeg[b] += 1
    queries = [next(it) - 1 for _ in range(qn)]
    start = [0] * n
    finish = duration[:]
    dq = deque(i for i in range(n) if indeg[i] == 0)
    processed = 0
    while dq:
        u = dq.popleft()
        processed += 1
        finish[u] = start[u] + duration[u]
        for v in adj[u]:
            if finish[u] > start[v]:

```

```

        start[v] = finish[u]
        indeg[v] -= 1
        if indeg[v] == 0:
            dq.append(v)
    if processed != n:
        print("IMPOSSIBLE")
    else:
        print(" ".join(str(finish[x]) for x in queries))

if __name__ == "__main__":
    main()

```

## 5.12 Common wrong answers

Using DFS reachability instead of topological sorting.  
 Updating start[v] with min instead of max.  
 Answering queries before detecting cycles.  
 Forgetting that independent modules can start at day 0.  
 Using int for long timelines in C++.

## 5.13 Tests that matter

No dependencies: finish[i] = duration[i].  
 A simple chain: finish accumulates all previous durations.  
 A diamond dependency: the join node waits for the slower branch.  
 A directed cycle: output IMPOSSIBLE.  
 A cycle disconnected from queried modules: still IMPOSSIBLE, because the problem says the dependencies contain a cycle.

## 6 C. One-Way Reversals

### 6.1 Restatement

There are directed roads. Traveling along a road's given direction costs nothing. Traveling against a road is allowed, but costs 1 reversal for that trip. Find the minimum number of reversals needed to travel from city 1 to city  $n$ , or print -1 if there is no possible route even after reversals.

### 6.2 Constraints and traps

$n$  and  $m$  can be up to  $2 * 10^5$ , so use an adjacency list and near-linear shortest path.

Important traps:

```
The original roads are directed.
Each input road creates two possible travel edges with different costs.
Ordinary BFS is wrong because cost 0 and cost 1 moves are not equal.
Dijkstra is correct but heavier than needed; 0-1 BFS is the intended tool.
The answer is -1 only if city  $n$  is unreachable in the underlying undirected
graph formed by the roads.
```

### 6.3 Observations

For every input road:

```
u -> v
```

we add:

```
u -> v cost 0
v -> u cost 1
```

Now every route in the original problem corresponds to a path in this weighted graph. The path cost is exactly the number of reversed roads used. Because all costs are 0 or 1, use 0-1 BFS.

### 6.4 Sample walkthrough

Input:

```
5 5
1 2
3 2
3 4
5 4
2 5
```

The route:

```
1 -> 2 -> 5
```

uses both roads in their original direction:

```
1 -> 2 cost 0
2 -> 5 cost 0
```

So the answer is 0.

Another small case:

```
3 2
2 1
3 2
```

To go from 1 to 3:

```
1 -> 2 reverses 2 -> 1, cost 1
2 -> 3 reverses 3 -> 2, cost 1
```

Answer: 2.

## 6.5 Algorithm

```
read n, m
adj = empty weighted adjacency list

for each road u, v:
    add edge u -> v with cost 0
    add edge v -> u with cost 1

dist[1] = 0
deque = [1]

while deque not empty:
    u = pop front
    for each edge u -> v with cost w:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
```

```

        if w == 0: push_front(v)
        else:      push_back(v)

if dist[n] is infinity: print -1
else: print dist[n]

```

## 6.6 Correctness proof

**Lemma 1.** Every valid trip in the original problem corresponds to a path in the constructed graph with the same number of reversals.

**Proof.** For each traveled road, if the trip follows the road direction, the constructed graph contains the cost-0 edge. If the trip goes against the road, the constructed graph contains the reverse cost-1 edge. Summing costs counts exactly the reversed roads.  $\square$

**Lemma 2.** Every path in the constructed graph corresponds to a valid trip in the original problem with the same reversal count.

**Proof.** Every constructed edge came from an original road. A cost-0 edge uses that road in its given direction; a cost-1 edge uses it against its direction and pays one reversal. Thus the graph path is a valid trip with equal cost.  $\square$

**Lemma 3.** 0-1 BFS computes shortest distances in the constructed graph.

**Proof.** When relaxing a cost-0 edge, the new state has the same distance as the current state, so it belongs at the front of the deque. When relaxing a cost-1 edge, the new state is one distance layer later, so it belongs at the back. This preserves nondecreasing distance processing, matching Dijkstra's order for edge weights 0 and 1.  $\square$

**Theorem.** The algorithm prints the minimum number of reversals needed to travel from city 1 to city  $n$ , or  $-1$  if no valid trip exists.

**Proof.** By Lemmas 1 and 2, minimizing reversals in the original problem is equivalent to finding a shortest path in the constructed 0/1 graph. By Lemma 3, 0-1 BFS computes that shortest distance. If the distance remains infinity, there is no path in the constructed graph, hence no valid trip.  $\square$

## 6.7 Complexity

Each original road creates two adjacency entries.

```

Time:   $O(n + m)$ 
Memory:  $O(n + m)$ 

```

## 6.8 C++ implementation notes

Use `deque<int>` and adjacency entries `(to, cost)`. The maximum answer is at most  $m$ , so `int` distances are enough here.

## 6.9 Python implementation notes

Use `collections.deque`. The graph has  $2m$  stored edges, so append tuples carefully but directly; no heap is needed.

## 6.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<vector<pair<int,int>>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        adj[u].push_back({v, 0});
        adj[v].push_back({u, 1});
    }
    const int INF = 1e9;
    vector<int> dist(n, INF);
    deque<int> dq;
    dist[0] = 0;
    dq.push_back(0);
    while (!dq.empty()) {
        int u = dq.front();
        dq.pop_front();
        for (auto [v, w] : adj[u]) {
            int nd = dist[u] + w;
            if (nd < dist[v]) {
                dist[v] = nd;
                if (w == 0) dq.push_front(v);
                else dq.push_back(v);
            }
        }
    }
    cout << (dist[n - 1] == INF ? -1 : dist[n - 1]) << '\n';
    return 0;
}
```

## 6.11 Full Python solution

```
from collections import deque
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    adj = [[] for _ in range(n)]
    for _ in range(m):
        u = next(it) - 1
        v = next(it) - 1
        adj[u].append((v, 0))
        adj[v].append((u, 1))
    inf = 10**9
    dist = [inf] * n
    dist[0] = 0
    dq = deque([0])
    while dq:
        u = dq.popleft()
        for v, w in adj[u]:
            nd = dist[u] + w
            if nd < dist[v]:
                dist[v] = nd
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    print(-1 if dist[n - 1] == inf else dist[n - 1])

if __name__ == "__main__":
    main()
```

## 6.12 Common wrong answers

Adding only the original directed cost-0 edges.  
Adding reverse edges with cost 0.  
Using BFS and counting number of roads instead of reversals.  
Printing 0 for unreachable cities.  
Using push\_back for both cost 0 and cost 1 relaxations.

## 6.13 Tests that matter

There is already a directed path from 1 to n -> answer 0.

All roads on the only path point backward -> answer equals path length.

City n is isolated -> -1.

There is a longer 0-cost route and a shorter 1-cost route -> choose 0-cost.

Parallel roads or repeated relaxations -> distance updates still work.

## 7 Testing and stress strategy

The section judge uses:

```
CP_TARGET=solution python3 sections/21_graphs_i_mixed_contest/check.py
```

The random tests target different risks:

Existing Network:

compare Kruskal behavior on connected, disconnected, already-connected, and duplicate-edge cases.

Build Timeline:

generate both DAGs and cyclic graphs; check no-edge and single-node cases.

One-Way Reversals:

compare 0-1 BFS answers on random directed graphs, including unreachable targets.

For personal stress testing, a brute force oracle is feasible on tiny graphs:

A: enumerate subsets of paid edges for  $n \leq 8$ .

B: for DAGs, compare against repeated relaxation in topological order; for cycles, compare against DFS cycle detection.

C: run ordinary Dijkstra on the constructed 0/1 graph and compare to 0-1 BFS.

## 8 What to carry forward

Mixed graph problems are mostly about modeling discipline.

```
Connect all vertices cheaply -> MST thinking.  
Respect prerequisites -> topological thinking.  
Minimize travel cost -> inspect edge weights before choosing BFS, 0-1 BFS, or  
Dijkstra.
```

In future sections, graph problems will combine these tools with dynamic programming, trees, and harder state spaces. The same diagnosis routine remains the first step.

## 9 D. Reliable Link

### 9.1 Restatement

Every undirected road has a difficulty. A route is judged by its hardest road, not by the sum of its road difficulties. We need the smallest possible value of that maximum on a route from city 1 to city  $n$ , or  $-1$  if no route exists. When  $n = 1$ , the empty route has difficulty 0.

This wording is a trap: it is not an ordinary shortest path. Adding a road of difficulty 4 and one of difficulty 5 gives route difficulty 5, not 9.

### 9.2 Threshold viewpoint

Fix a candidate limit  $X$ . Keep only roads with difficulty at most  $X$ .

```
There is a route of difficulty at most X
if and only if
1 and n are connected using only roads of difficulty at most X.
```

As  $X$  increases, roads only get added, so connectivity can only change from false to true. Sort all roads by difficulty and add them in that order with a DSU. The first difficulty at which cities 1 and  $n$  share a component is the answer.

This is related to Kruskal, but we do not need to finish an MST. Stop as soon as the two required cities become connected.

### 9.3 Walk-through

For these roads:

```
1-3: 3    1-2: 4    3-4: 5    4-5: 5    2-5: 8
```

Process increasing difficulties:

```
add 1-3 (3): components of 1 and 5 differ
add 1-2 (4): components of 1 and 5 differ
add 3-4 (5): components of 1 and 5 differ
add 4-5 (5): 1 and 5 are now connected
```

The answer is 5. The connecting route is  $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ , whose maximum edge difficulty is 5.

## 9.4 Algorithm

```
if n is 1:
    print 0
    stop

sort all roads by difficulty
create DSU with one component per city

for each road (u, v, w) in sorted order:
    union(u, v)
    if find(1) equals find(n):
        print w
        stop

print -1
```

## 9.5 Correctness proof

**Lemma 1.** After all roads of difficulty at most  $x$  have been processed, the DSU puts two cities in the same component exactly when they are connected using only roads of difficulty at most  $x$ .

**Proof.** DSU starts with isolated cities and merges exactly the endpoints of every processed road. Thus every DSU merge represents an allowed road path, and every allowed road is included in the merges. Its components are exactly the connected components of the threshold graph.  $\square$

**Lemma 2.** When the algorithm first finds cities 1 and  $n$  connected at road difficulty  $w$ , there exists a route from 1 to  $n$  whose maximum road difficulty is at most  $w$ .

**Proof.** All roads processed so far have difficulty at most  $w$ . By Lemma 1, DSU connectivity gives a path using only those roads.  $\square$

**Lemma 3.** No route from 1 to  $n$  has maximum road difficulty less than the first reported value  $w$ .

**Proof.** Before processing roads of difficulty  $w$ , the two cities were in different DSU components. By Lemma 1, they were disconnected in the graph of all roads with difficulty below  $w$ . Such a route therefore cannot exist.  $\square$

**Theorem.** The algorithm prints the required minimum route difficulty, or  $-1$  exactly when city  $n$  is unreachable from city 1.

**Proof.** Lemmas 2 and 3 show that a reported  $w$  is feasible and minimal. If the scan ends without connecting the cities, Lemma 1 says no input road path connects them, so  $-1$  is correct. The  $n = 1$  case is correct by the empty route convention.  $\square$

## 9.6 Complexity

```
Time:    O(m log m)
Memory: O(n + m)
```

Sorting roads dominates; DSU operations are effectively constant amortized time.

## 9.7 Implementation notes

- Use `long long` for road difficulties in C++ even though DSU stores city ids.
- Repeated roads are harmless; DSU ignores a road within an existing component.
- Do not add difficulties along a path.
- Do not return the first road incident to city `n`; check DSU connectivity.

## 9.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> parent, size;
    explicit DSU(int n) : parent(n + 1), size(n + 1, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        return parent[x] == x ? x : parent[x] = find(parent[x]);
    }
    void unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return;
        if (size[a] < size[b]) swap(a, b);
        parent[b] = a;
        size[a] += size[b];
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<tuple<long long, int, int>> roads;
    for (int i = 0; i < m; ++i) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        roads.emplace_back(w, u, v);
    }
```

```

if (n == 1) {
    cout << 0 << "\\n";
    return 0;
}
sort(roads.begin(), roads.end());
DSU dsu(n);
for (auto [w, u, v] : roads) {
    dsu.unite(u, v);
    if (dsu.find(1) == dsu.find(n)) {
        cout << w << "\\n";
        return 0;
    }
}
cout << -1 << "\\n";
}

```

## 9.9 Full Python solution

```

import sys

class DSU:
    def __init__(self, n: int) -> None:
        self.parent = list(range(n + 1))
        self.size = [1] * (n + 1)

    def find(self, x: int) -> int:
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a: int, b: int) -> None:
        a = self.find(a)
        b = self.find(b)
        if a == b:
            return
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    roads = sorted(
        (data[i + 2], data[i], data[i + 1])

```

```

        for i in range(2, 2 + 3 * m, 3)
    )
    if n == 1:
        print(0)
        return

    dsu = DSU(n)
    for w, u, v in roads:
        dsu.union(u, v)
        if dsu.find(1) == dsu.find(n):
            print(w)
            return
    print(-1)

if __name__ == "__main__":
    main()

```

## 9.10 Common wrong answers

- Summing road difficulties and using ordinary shortest-path Dijkstra.
- Building a full MST even after cities 1 and  $n$  are connected; correct but unnecessary.
- Declaring  $-1$  merely because a low-difficulty prefix is disconnected.
- Forgetting the one-city empty-route case.

## 9.11 Tests that matter

- city 1 and city  $n$  disconnected;
- one city and no roads;
- a low-sum route with one very difficult road versus a longer route with a lower maximum;
- repeated roads with different difficulties;
- many equal-difficulty roads that connect the endpoints together.

## 10 Stress testing

For small graphs, use a different formulation: minimax Dijkstra. Let `dist[v]` be the smallest possible maximum edge on a route to `v`, and relax an edge of difficulty `w` with:

```
candidate = max(dist[u], w)
```

Use a heap on that candidate and compare its answer with the DSU threshold sweep. The local generator uses this independent oracle for small random graphs and a direct-answer 200000-edge scale case.

# 11 What to carry forward

When a route is scored by its worst edge, convert a minimization question into a connectivity threshold:

```
"Can I travel using only edges  $\leq X$ ?"
```

Sorting thresholds and DSU are often simpler than forcing an additive shortest path interpretation.

## 12 E. Station Coverage Report

### 12.1 Restatement and model

For every vertex, imagine asking for its distance to the closest station. After obtaining those distances, report how many vertices have no station in their component and, among the rest, the farthest vertex with a smallest-index tie break.

Running a separate BFS from every station would repeat most graph work. The key recognition is that “distance to the closest of several sources” is one multi-source shortest-path computation.

### 12.2 Multi-source BFS

Put every distinct station in the queue initially with distance zero. This is equivalent to adding a virtual source joined to every station by a zero-length edge, except that no explicit vertex is needed. Ordinary BFS then expands all distance-zero stations, followed by all distance-one vertices, and so on.

Duplicate station entries must not be enqueued twice. Testing whether its distance is still  $-1$  handles this naturally.

### 12.3 Algorithm

1. Build the undirected adjacency list.
2. Initialize all distances to  $-1$ .
3. For each station not initialized before, set its distance to zero and push it into the queue.
4. Run BFS. On the first visit to  $v$ , assign  $\text{distance}[v] = \text{distance}[u] + 1$ .
5. Scan vertices in increasing index:
  - count every distance  $-1$  as unreachable;
  - otherwise replace the farthest answer only when the distance is strictly larger.

The strict comparison in the final scan preserves the smallest vertex on ties.

### 12.4 Correctness proof

Initially, precisely the stations have distance zero, which is their true distance to the nearest station. BFS processes vertices in nondecreasing distance. When an unvisited neighbor  $v$  is reached from  $u$ , there is a route of length  $\text{distance}[u] + 1$  from a station to  $v$ . Any shorter route would have reached  $v$  from an earlier BFS layer, contradicting that it was unvisited. Thus every assigned distance is the minimum distance to any station.

Vertices left at -1 were never reachable from any initial source, so the unreachable count is correct. The increasing-index scan chooses the maximum finite distance and retains the first vertex attaining it, exactly enforcing the requested tie break.

## 12.5 Complexity and pitfalls

Every vertex enters the queue at most once and every undirected edge is examined twice. Time is  $O(n+m)$  and memory is  $O(n+m)$ .

Common mistakes are running one BFS per station, forgetting disconnected components, enqueueing duplicate stations repeatedly, or using  $\geq$  in the final scan and thereby selecting the largest tied vertex.

## 12.6 Tests that matter

```
One station in a connected path -> ordinary BFS distances.
Stations at both ends of a path -> nearest-source boundary and tie break.
Repeated station indices -> same result as one occurrence.
A component with no station -> all its vertices are unreachable.
Every vertex is a station -> farthest distance zero, vertex 1.
```

## 12.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;
    vector<vector<int>> graph(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        --u; --v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }
    vector<int> distance(n, -1);
    queue<int> queue;
    while (k--) {
        int station;
        cin >> station;
        --station;
```

```

        if (distance[station] == -1) {
            distance[station] = 0;
            queue.push(station);
        }
    }
    while (!queue.empty()) {
        int u = queue.front();
        queue.pop();
        for (int v : graph[u]) if (distance[v] == -1) {
            distance[v] = distance[u] + 1;
            queue.push(v);
        }
    }
    int unreachable = 0, vertex = -1, farthest = -1;
    for (int i = 0; i < n; ++i) {
        if (distance[i] == -1) {
            ++unreachable;
        } else if (distance[i] > farthest) {
            farthest = distance[i];
            vertex = i;
        }
    }
    cout << unreachable << ' ' << vertex + 1 << ' ' << farthest << '\n';
}

```

## 12.8 Full Python solution

```

import sys
from collections import deque

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m, k = data[:3]
    graph = [[] for _ in range(n)]
    index = 3
    for _ in range(m):
        u, v = data[index] - 1, data[index + 1] - 1
        index += 2
        graph[u].append(v)
        graph[v].append(u)
    distance = [-1] * n
    queue = deque()
    for station in data[index:index + k]:
        station -= 1
        if distance[station] == -1:
            distance[station] = 0
            queue.append(station)

```

```
while queue:
    u = queue.popleft()
    for v in graph[u]:
        if distance[v] == -1:
            distance[v] = distance[u] + 1
            queue.append(v)
unreachable = distance.count(-1)
farthest = max(distance)
vertex = distance.index(farthest) + 1
print(unreachable, vertex, farthest)

if __name__ == "__main__":
    main()
```

## 13 F. Component Repair

### 13.1 Restatement and observations

We need three facts about an undirected graph: its number of connected components, the minimum number of roads that can connect the whole graph, and the largest component size.

A DFS or BFS started from an unvisited vertex discovers exactly one component. If the graph has  $c$  components, one new road can merge at most two of them and therefore reduce the component count by at most one. At least  $c-1$  roads are necessary. They are also sufficient: choose one representative per component and connect the representatives in a chain.

### 13.2 Algorithm

1. Build the undirected adjacency list.
2. Scan all vertices.
3. Whenever an unseen vertex is found:
  - increment the component count;
  - run iterative DFS from it;
  - count every popped vertex to obtain this component's size;
  - update the largest size.
4. Print `components`, `components - 1`, and `largest`.

Mark a vertex when pushing it, not when popping it. Otherwise several neighbors may push the same vertex and inflate memory usage.

### 13.3 Correctness proof

Each DFS follows exactly graph edges, so every vertex it reaches belongs to the start vertex's component. Conversely, every vertex in that component has a path from the start; following the path inductively shows DFS eventually reaches it. Hence one search visits exactly one complete component.

The outer scan starts a search precisely for components not visited before, so the number of searches equals the number of components, and each recorded size is exact. Taking their maximum gives the largest component.

Finally, connecting  $c$  initial components needs at least  $c-1$  roads because each road reduces their count by at most one. A chain through one vertex from each component uses exactly  $c-1$  roads and makes all components connected. Therefore the printed repair count is minimal.

## 13.4 Complexity and pitfalls

Time is  $O(n+m)$  and memory is  $O(n+m)$ . Iterative DFS avoids recursion-depth failures on a path of length 200000.

Do not answer with  $n-1$ ; existing roads already connect vertices. Isolated vertices are components of size one. A graph that is already connected needs zero new roads.

## 13.5 Tests that matter

```
n = 1, m = 0          -> 1 0 1
No edges              -> n, n-1, 1
Already connected     -> 1 0 n
Several unequal components -> verifies both count and maximum size
Repeated edges and cycles -> must not change component membership
```

## 13.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> graph(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        --u; --v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }
    vector<char> seen(n);
    int components = 0, largest = 0;
    for (int start = 0; start < n; ++start) if (!seen[start]) {
        ++components;
        int size = 0;
        vector<int> stack{start};
        seen[start] = true;
        while (!stack.empty()) {
            int u = stack.back();
            stack.pop_back();
            ++size;
            for (int v : graph[u]) if (!seen[v]) {
                seen[v] = true;
                stack.push_back(v);
            }
        }
        largest = max(largest, size);
    }
    cout << components << " " << largest << endl;
}
```

```

        stack.push_back(v);
    }
}
largest = max(largest, size);
}
cout << components << ' ' << components - 1 << ' ' << largest << '\n';
}

```

## 13.7 Full Python solution

```

import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    graph = [[] for _ in range(n)]
    index = 2
    for _ in range(m):
        u, v = data[index] - 1, data[index + 1] - 1
        index += 2
        graph[u].append(v); graph[v].append(u)
    seen = [False] * n
    sizes = []
    for start in range(n):
        if seen[start]:
            continue
        seen[start] = True
        stack = [start]
        size = 0
        while stack:
            u = stack.pop()
            size += 1
            for v in graph[u]:
                if not seen[v]:
                    seen[v] = True
                    stack.append(v)
            sizes.append(size)
    print(len(sizes), len(sizes) - 1, max(sizes))

if __name__ == "__main__":
    main()

```

## 14 G. Alternating Road Route

### 14.1 Why a vertex alone is not enough state

The cheapest way to reach vertex  $u$  is not always the best prefix for future moves. If it ends with a red road, the next road must be blue; a route ending with blue permits a different set of transitions. Therefore the state must remember the last color:

```
(u, R) = at u, with the previous road red
(u, B) = at u, with the previous road blue
```

This is the same state-expansion technique used for parity, coupons, or a small automaton attached to a graph.

### 14.2 Expanded graph and algorithm

Each original road  $u \rightarrow v$  of color  $c$  creates transitions to  $(v, c)$  only from states whose last color is not  $c$ . Transition cost remains the road cost. Because all costs are nonnegative, Dijkstra applies to this expanded graph.

The first road may have either color. The implementation handles this by relaxing every outgoing road of vertex 1 as an initial state. An alternative is a third synthetic “no previous color” state. If  $n = 1$ , the empty route has cost zero and needs explicit handling with the first formulation.

Steps:

1. Build the directed colored adjacency list.
2. Set both distances per vertex to infinity.
3. Insert all one-road routes leaving vertex 1.
4. Pop  $(cost, u, last)$  from the min-heap, ignoring stale entries.
5. Relax only edges whose color differs from  $last$ .
6. Return the smaller distance to  $(n, R)$  and  $(n, B)$ , or  $-1$ .

### 14.3 Correctness proof

Map every valid original route to the sequence of expanded states recording the color of each used road. Consecutive colors differ, so every step is an expanded-graph edge, and the total cost is unchanged. Conversely, every path in the expanded graph uses only edges of a color different from its stored previous color, so forgetting the color component yields a valid alternating route of the same cost. This is a cost-preserving correspondence.

Dijkstra finds the cheapest expanded path to every state because expanded edge costs are nonnegative. Taking the minimum of the two destination states therefore gives exactly the cheapest alternating route to vertex  $n$ .

## 14.4 Complexity and common mistakes

There are  $2n$  states and at most  $2m$  usable state transitions. Time is  $O((n+m) \log n)$  and memory is  $O(n+m)$ .

- The roads are directed; do not add a reverse edge.
- Store two distances per vertex, not one.
- The restriction concerns consecutive colors, not total counts of colors.
- Use 64-bit costs: a route sum can exceed  $10^9$ .
- Zero-cost edges are valid and remain compatible with Dijkstra.

## 14.5 Tests that matter

```
n = 1                                -> 0
Only one direct road                 -> its cost
Cheapest ordinary path repeats a color -> must be rejected
Two routes reaching u with different last colors -> both states matter
Destination unreachable under alternation -> -1
Parallel roads with different colors and zero costs
```

For small graphs, the randomized oracle can run Bellman-Ford or Dijkstra on the explicit  $2n$ -state graph.

## 14.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    struct Edge { int to, color; ll cost; };
    vector<vector<Edge>> graph(n);
    while (m--) {
        int u, v;
        ll cost;
        char color;
        cin >> u >> v >> cost >> color;
        graph[u - 1].push_back({v - 1, color == 'R' ? 0 : 1, cost});
    }
    const ll INF = (1LL << 62);
    vector<array<ll, 2>> dist(n, {INF, INF});
    using State = tuple<ll, int, int>;
```

```

priority_queue<State, vector<State>, greater<State>> heap;
for (int color = 0; color < 2; ++color) {
    for (auto edge : graph[0]) if (edge.color == color) {
        if (edge.cost < dist[edge.to][color]) {
            dist[edge.to][color] = edge.cost;
            heap.push({edge.cost, edge.to, color});
        }
    }
}
if (n == 1) {
    cout << 0 << '\n';
    return 0;
}
while (!heap.empty()) {
    auto [cost, u, last] = heap.top();
    heap.pop();
    if (cost != dist[u][last]) continue;
    for (auto edge : graph[u]) if (edge.color != last) {
        ll next = cost + edge.cost;
        if (next < dist[edge.to][edge.color]) {
            dist[edge.to][edge.color] = next;
            heap.push({next, edge.to, edge.color});
        }
    }
}
ll answer = min(dist[n - 1][0], dist[n - 1][1]);
cout << (answer == INF ? -1 : answer) << '\n';
}

```

## 14.7 Full Python solution

```

import heapq
import sys

def main():
    tokens = sys.stdin.buffer.read().split()
    n, m = map(int, tokens[:2])
    graph = [[] for _ in range(n)]
    index = 2
    for _ in range(m):
        u = int(tokens[index]) - 1
        v = int(tokens[index + 1]) - 1
        cost = int(tokens[index + 2])
        color = 0 if tokens[index + 3] == b"R" else 1
        index += 4
        graph[u].append((v, cost, color))
    if n == 1:

```

```

        print(0)
        return
    inf = 10**30
    dist = [[inf, inf] for _ in range(n)]
    heap = []
    for v, cost, color in graph[0]:
        if cost < dist[v][color]:
            dist[v][color] = cost
            heapq.heappush(heap, (cost, v, color))
    while heap:
        cost, u, last = heapq.heappop(heap)
        if cost != dist[u][last]:
            continue
        for v, weight, color in graph[u]:
            if color == last:
                continue
            new = cost + weight
            if new < dist[v][color]:
                dist[v][color] = new
                heapq.heappush(heap, (new, v, color))
    answer = min(dist[-1])
    print(-1 if answer == inf else answer)

if __name__ == "__main__":
    main()

```